

Document control

What this document is, who it is for, and how to read a hundred pages without losing the thread.

00.1 Purpose

This is the single source of truth for **The Everything Platform**, the working name for RMH Studios' first product. It is written to do three jobs at once: to make the founder think clearly before writing code, to give a collaborator or early hire enough context to be useful on day one, and to be the document you hand a potential partner, advisor, or grant committee when they ask "what exactly are you building, and how?" Everything here is meant to be acted on, not admired.

00.2 How to read this

The document moves from *why* to *what* to *how* to *when*. Sections 01–04 set up the studio, the problem, and the thesis. Sections 05–09 specify the product itself, module by module, down to acceptance criteria. Sections 10–13 are the plan: a \$200 budget, a 24-month build calendar with a month-by-month log, and the milestones that gate each phase. Sections 14–20 cover going to market, pricing, growth, money, risk, and the appendices you'll actually reference.

READING NOTE

The 24-month timeline in Section 12 is the operational heart of this plan. If you read nothing else, read that and the budget in Section 11. Everything before it is the case for why those two sections look the way they do.

00.3 A word on the budget

The cash budget for this plan is **\$200**. That is not a typo and it is not a placeholder. It is a deliberate constraint that shapes every decision downstream: the stack is built on free tiers, growth is earned rather

than bought, the team is one person plus the open-source commons, and profitability — not a funding round — is the only runway that matters. Where money is spent, it is named to the dollar. The discipline of \$200 is treated as a feature of the plan, because it forces the product to be good enough that people tell each other about it.

00.4 Revision history

VERSION	DATE	AUTHOR	SUMMARY OF CHANGE
v0.1	M01 · wk1	Founder	Initial thesis, problem framing, rough scope.
v0.4	M01 · wk3	Founder	Module catalog drafted; budget set at \$200.
v0.6	M02	Founder	24-month timeline added; metrics defined.
v0.8	M03	Founder	GTM, pricing, financial model added.
v0.9	current	Founder	Full consolidation; ready for first external read.

LIVING DOCUMENT

This plan is expected to be wrong in specifics and right in shape. Re-read it at the end of each quarter (Sections 13.x mark the checkpoints) and rewrite the parts the last three months proved false. A plan that never changes was never a plan.

Contents

Twenty sections across roughly one hundred pages.
Page numbers are approximate to print pagination.

00	Document control	front-matter	p.002
01	Executive summary	the whole plan in two pages	p.004
02	The studio	who RMH Studios is and what it stands for	p.005
03	The problem	the tax builders pay for fragmented tools	p.008
04	The thesis	why one platform, why now	p.012
05	Users & personas	who this is for, in detail	p.015
06	Market landscape	the field and where we fit	p.019
07	Product principles	the rules the product obeys	p.023
08	Architecture	how the thing is actually built	p.026
09	Module catalog	every part of the platform, specified	p.031
10	Design system	the visual and interaction language	p.052
11	The \$200 budget	every dollar, accounted for	p.055
12	24-month timeline	the build calendar, month by month	p.060
13	Milestones & gates	what unlocks the next phase	p.084
14	Go-to-market	how people find and adopt it	p.087
15	Pricing & monetization	how it makes money	p.090
16	Growth model	the loops that compound	p.097

16	Growth model	the loops that compound	p.072
17	Metrics & KPIs	what we measure and why	p.094
18	Financial projections	the bootstrap math	p.096
19	Risk register	what could go wrong, and the plan	p.098
20	Appendices	checklists, glossary, references	p.100

§01 / SUMMARY

Executive summary

The whole plan in two pages. If a reader stops here, this is what they should walk away believing.

Builders live in two worlds that refuse to talk to each other. There is the world of code — repositories, branches, tickets, deploys — and the world of craft — files, canvases, drafts, references, feedback. Almost everyone who makes things for a living moves between both every day, and pays a tax every time they cross the border: context lost, versions forgotten, work duplicated, the thread of a project scattered across eight tabs.

The Everything Platform is one workspace where both worlds share the same project, the same history, and the same people. A project holds your code and your design files and your docs and your tasks together. A change to a mockup and a change to a component live in the same timeline. The brief, the build, and the ship are one continuous thread rather than three disconnected tools that you reconcile by hand.

TARGET USER	CORE UNIT	CASH BUDGET	NORTH STAR
Builder coder + maker	Project code + craft, one home	\$200 24-month total	WAB weekly active builders

01.1 The shape of the plan

RMH Studios builds this as a bootstrapped product, solo, over 24 months. The first six months produce a sharp, narrow v1 — projects, unified files, version history, and a single killer feature that no fragmented stack can match. Months 7–12 turn early users into a habit

and open a paid tier. Months 13–18 widen the platform with the modules that make it sticky for small teams. Months 19–24 turn habit into a business that pays the founder a real salary without a single dollar of outside capital.

01.2 Why it can work on \$200

The platform is a thin, well-designed layer over infrastructure that is free until you have real usage: edge hosting, a generous database free tier, object storage that bills in pennies, and an open-source-first toolchain. The expensive part of most startups — acquiring users — is replaced by building in public, shipping a genuinely useful free tier, and letting a product that respects builders earn its own word of mouth. The \$200 is spent on the few things that genuinely cost money: a domain, a little storage overage, and the occasional paid month of a service the free tier outgrows.

01.3 What success looks like

By month 24, success is not a valuation. It is a product that several thousand people open every week, a few hundred of whom pay for it, generating enough monthly revenue to cover costs many times over and pay the founder a living wage. From there, the choice to raise money, hire, or stay small is a choice made from strength rather than necessity.

THE ONE-SENTENCE VERSION

The Everything Platform unifies code and creative work into a single project workspace, shipped solo on a \$200 budget over 24 months, aiming for a self-sustaining business built on a free tier people love and a paid tier small teams happily buy.

§02 / STUDIO

The studio

Before the product, the point of view. RMH Studios exists because the tools for people who both code and create were never designed to be used by the same person on the same day.

02.1 What RMH Studios is

RMH Studios is a one-person product studio with a long-game ambition: to build the software layer that serves people who refuse to choose between being technical and being creative. The studio is not a marketing agency, not a dev shop for hire, and not a content brand. It is a maker of products, and The Everything Platform is the first and largest of them. The studio's bet is that the line between "developer tools" and "creative tools" is an accident of how the software industry grew up, not a real boundary in how people work.

02.2 Mission

MISSION

Give builders one home for everything a project is — the code, the craft, the conversation, and the history — so the work, not the tooling, is where the effort goes.

02.3 Beliefs

// 01

The split is artificial

Treating "engineering" and "design" as

// 02

Context is the product

The value isn't another editor. It's keeping the

// 03

Small can win

A focused team of one, building in public on a tiny budget, can

separate worlds
with separate
tools is a
historical
accident. Most
real projects are
both at once.

why, the what,
and the history
of a project in
one place so
nothing has to
be re-explained.

out-care a
funded
competitor
optimizing for a
different game.

02.4 Operating principles

- **Ship weekly, in public.** Visible progress is the cheapest marketing a \$200 budget can buy, and the discipline keeps scope honest.
- **Free tier first.** The free tier is the product. The paid tier is a convenience for people who already love it.
- **Default to less.** Every feature must earn its place against the cost of the next person having to understand it.
- **Profit is the runway.** No outside money until — and unless — it is a choice made from strength.
- **Respect the builder.** No dark patterns, no engagement bait, no selling attention. The user is the customer, never the product.

02.5 Why "RMH Studios"

The studio name is intentionally a person's house, not a category. "Studios" signals a place where things are made across disciplines — a recording studio, a design studio, a game studio all share the word — which is exactly the spirit of a product for people who make many kinds of things. The product name, *The Everything Platform*, is a working title that doubles as a promise and a stress test: if a feature wouldn't belong in "everything a project needs," it probably doesn't belong at all.

02.6 The founder's constraints, honestly

A plan that ignores the realities of who is executing it is fiction. This plan is built for one person with limited time, no capital, and a need to see revenue before it can justify going full time. Those constraints are written into every section that follows.

CONSTRAINT	REALITY	HOW THE PLAN RESPONDS
Time	Part-time at first; nights and weekends.	Scope each month to ~40–60 focused hours; cut ruthlessly.
Money	\$200 total cash budget.	Free-tier stack; spend only on domain, storage, occasional overage.
People	Solo, no co-founder yet.	Lean on open source, templates, and community; design for one maintainer.
Distribution	No audience on day one.	Build in public; earn the first 100 users by hand.
Risk	Cannot afford a year of zero feedback.	Ship something usable by month 4; learn in the open.

02.7 What the studio will not do

Saying no is how a one-person studio survives. RMH Studios will not chase enterprise sales in the first two years, will not build a mobile-native app before the web product is loved, will not add AI features for their own sake, and will not take on consulting work that starves the product of time. Each of these is a reasonable business — none of them is *this* business.

DEFINITION

Throughout this document, a **builder** means a person who both writes code and produces creative work — a designer who codes, an engineer who designs, an indie hacker, a technical artist, a founder who ships their own product. The platform is for them first.

§03 / PROBLEM

The problem

Builders pay a quiet, constant tax for living in two toolchains at once. This section names that tax precisely, because a problem you can measure is a problem you can sell the solution to.

03.1 The border tax

Consider an ordinary afternoon for someone building their own product. They sketch a screen in a design tool, export an image, drop it into a chat to get feedback, open a ticket in a project tracker describing the change, switch to their editor to implement it, commit the code to a repository, write a note in a separate doc about why they made a decision, and update a spreadsheet tracking what's left. Eight tools. Each crossing loses a little context. By the end of the week, the *why* behind half the decisions lives only in their head.

This is the border tax: the cumulative cost of moving work, context, and attention across tools that were never designed to share a project. It is rarely catastrophic and always present, which is exactly why it is under-served — no single crossing hurts enough to fix, but the sum is enormous.

CROSSING	WHAT'S LOST	FREQUENCY	FELT COST
Design → code	Intent, spec, exact values	Many times/day	Re-derivation, drift
Code → task tracker	Why a change happened	Daily	Stale tickets, lost rationale
Chat → everywhere	Decisions made in passing	Constant	Undocumented choices
File → file	Which version is current	Daily	Wrong-version mistakes

Tool → tool (auth)	Time, focus, momentum	Dozens/day	Context-switch fatigue
-----------------------	--------------------------	------------	---------------------------

THE INSIGHT

The pain isn't any one tool being bad. Most of them are excellent.
The pain is the *spaces between them* — and no one owns the
spaces between.

03.2 Who feels it most

The tax falls hardest on people who do not have a team to absorb it. A large company can hire a project manager to keep tickets in sync and a design-ops person to maintain the handoff. A solo builder or a two-person team *is* the project manager and the design-ops person, on top of doing the actual work. The smaller the team, the larger the relative tax — which is precisely the market a bootstrapped product can serve before anyone bigger bothers to.

03.3 Why existing tools don't solve it

Dev tools ignore craft

Repos, issue trackers, and CI assume the artifact is text. Images, mockups, and creative drafts are second-class at best — attachments, not first-class objects with their own history.

Design tools ignore code

Creative tools treat code as an export target, not a living part of the project. The handoff is a wall, not a door.

PM tools sit above both

Project trackers describe work without containing it. The ticket points at the work; it isn't the work. Context still scatters.

"All-in-one" tools are docs

The popular all-in-one workspaces are excellent at documents and databases, but code and creative files are guests, not residents.

03.4 The gap, stated plainly

No tool treats a project's code and its creative work as equal citizens of the same home, with one shared history and one shared cast of people. That gap is the product.

03.5 Why now

- **Infrastructure got cheap.** Edge hosting, generous database free tiers, and pennies-per-gigabyte storage make it possible to run a real product for almost nothing until it has real usage.
- **The "build in public" norm is mature.** A solo founder can earn an audience by shipping openly — a distribution channel that didn't exist at this scale a decade ago.
- **The maker population exploded.** The number of people who both code and create — indie hackers, technical designers, solo founders — has grown faster than the tools serving them.
- **Tool fatigue is real.** People are actively tired of stitching ten subscriptions together and are looking for consolidation they can trust.

The thesis

One sentence, then the argument that holds it up.

THESIS

If you make code and creative work first-class citizens of the same project, with one shared history and one shared set of people, you remove the largest hidden cost in a builder's day — and a product that does this well, given away generously and sold modestly, can become a self-sustaining business built by one person.

04.1 The unit of value: the project

Everything in the platform orbits a single object: the **project**. A project is not a folder and not a repo and not a board. It is the home for everything a piece of work is — its code, its design files, its documents, its tasks, its conversations, and the complete timeline of how all of those changed together. When you open a project, you see the whole of it, not a slice filtered through one tool's worldview.

04.2 The wedge: unified history

A new platform cannot win by being a worse version of ten tools at once. It wins by doing one thing those ten tools structurally cannot: showing a **single, unified timeline** where a change to a design and a change to the code that implements it sit next to each other, attributed to the same person, tied to the same decision. That timeline is the wedge — the one feature worth switching for — and the rest of the platform is built outward from it.

LAYER	WHAT IT IS	WHY IT MATTERS
Project	The container for all work	One home, one mental model

Timeline	Unified history of every change	The wedge — nothing else has it
Modules	Code, files, tasks, docs, chat	Where the work actually happens
People	Shared cast across all modules	Attribution and context, everywhere

04.3 The strategy: narrow, then wide

The platform launches narrow on purpose. v1 is projects, unified files, version history, and the timeline — nothing more. It is better to be loved by a hundred people for one true thing than tolerated by a thousand for ten mediocre ones. Width comes only after the narrow core has earned a habit. Section 12's timeline encodes this discipline month by month.

04.4 Why a bootstrapper can win this

A funded competitor optimizes for the metrics that justify the next round: seats, logos, expansion revenue. That pressure pushes them toward enterprise features and away from the lonely solo builder who pays \$8 a month. A bootstrapped studio has the opposite incentive — it can serve exactly the under-served maker that the funded world ignores, and do it with a level of care that doesn't survive a growth-at-all-costs mandate. The constraint of \$200 is, paradoxically, a strategic moat: it forces a product so good that it spreads on its own, because there is no other way to grow.

04.5 Falsifiable bets

A thesis you can't disprove is a hope. Here are the bets this plan is willing to be wrong about, and how each is tested.

BET	IF TRUE, WE'LL SEE...	TESTED BY
Unified history is a real wedge	Users cite the timeline as the reason they stay	M04–M06 user interviews
Solo builders will pay	Free→paid conversion > 2% by M12	Paid tier launch, M07
Build-in-public earns users	First 500 users with \$0 ad spend	M01–M09 growth log
One person can maintain it	No burnout, weekly ships sustained	Quarterly founder check-in
Free tier won't bankrupt us	Infra stays under budget at 5k users	M11 cost review

KILL CRITERIA

If, by month 9, fewer than 300 people use the product weekly and the unified timeline is not the thing they mention first, the thesis is wrong and the plan should pivot the wedge — not double down. Honesty here is cheaper than a wasted year.

Users & personas

Five real shapes of person, so that "the builder" stops being an abstraction and starts being someone you can picture using the thing.

M

Maya — the solo founder

PRIMARY · INDIE HACKER

Building her own SaaS at nights. She designs the UI, writes the React, manages her own roadmap, and posts progress on social. She currently juggles a design tool, a repo, a notes app, and a kanban board, and loses decisions constantly. She is the single most important person this product is for: if Maya doesn't love it, nothing else matters.

Needs: one place for her whole project · history she can trust · zero setup tax · a free tier that's actually usable.

D

Devin — the technical designer

PRIMARY · DESIGN-ENGINEER

Works at a small startup, lives in the seam between design and front-end. Spends half his day translating mockups into components and the other half explaining why they drifted. The unified timeline is, for him, the killer feature — it ends the "which version is current" argument forever.

Needs: design and code side by side · clear change attribution · feedback in context, not in a separate chat.

Priua — the two-person studio

P

SECONDARY · SMALL TEAM

Runs a tiny product studio with one collaborator. They take on client builds that are equal parts brand and software. They need a shared home for each client project and a clean way to show progress. She is the first paying customer profile — small teams who feel the tax doubled.

Needs: shared projects · simple roles · a presentable timeline to show clients · predictable, small pricing.

L

Leo — the learning builder

TERTIARY • STUDENT / HOBBYIST

Learning to code and design at the same time, building side projects to figure out what he likes. Money is tight; the free tier is the only tier he'll touch for a long time. He matters because today's learner is tomorrow's Maya, and because his enthusiasm is free distribution.

Needs: a generous free tier • gentle onboarding • templates to start from • a community to learn alongside.

R

Renata — the creative technologist

TERTIARY • ARTIST WHO CODES

Makes interactive art and generative work. Her "code" and her "creative output" are genuinely the same thing, which is why fragmented tools frustrate her most. She is a vocal, influential edge case — the kind of user whose enthusiasm reaches exactly the audience RMH wants.

Needs: code and visual output in one timeline • easy sharing of finished pieces • a portfolio surface that doesn't feel corporate.

05.1 Priority and why

PERSONA	PRIORITY	PAYS?	ROLE IN THE PLAN
Maya • solo founder	P0	Eventually	The product is designed around her first.
Devin • design-engineer	P0	Maybe (team)	Proves the timeline wedge is real.
Priya • small studio	P1	Yes, first	The first reliable revenue.

Leo · learner	P2	Rarely	Free-tier love; future Mayas; word of mouth.
Renata · creative tech	P2	Sometimes	Influence and credibility at the edge.

DESIGN RULE

When two personas want different things, Maya wins, then Devin. The product is allowed to be imperfect for Priya's team needs in year one. It is not allowed to be imperfect for the solo builder, ever.

05.2 Jobs to be done

Personas describe *who*; jobs describe *what they're trying to get done*. The platform earns its place by doing these jobs better than the stitched-together alternative.

WHEN I...	I WANT TO...	SO THAT...
start a new project	set up one home for code, files, and tasks in a minute	I can begin building instead of configuring
change a design	see the code it affects in the same timeline	nothing drifts out of sync
make a decision	capture the why next to the work	future-me isn't confused
get feedback	collect it in context on the actual artifact	comments don't scatter across chat
ship something	share a clean view of progress	clients and followers can see the work
come back after a week	read the timeline and remember everything	I lose no momentum

05.3 Anti-personas

Equally important: who this is *not* for, at least in year one. Naming them keeps the roadmap honest.

- **The 200-person engineering org.** They have process, tooling, and budget this product won't serve well for years. Chasing them now would distort everything.
- **The pure project manager.** Someone who never touches the work itself wants a reporting tool, not a workspace. That's a different product.
- **The enterprise procurement buyer.** SSO, audit logs, and compliance checklists are a year-three conversation, not a year-one one.

§06 / MARKET

Market landscape

The neighborhoods this product borders, who lives in each, and why there's room for a small new house on the corner.

06.1 The adjacent categories

CATEGORY	EXAMPLES (TYPE)	WHAT THEY NAIL	WHERE THEY LEAVE A GAP
Dev platforms	Code hosting + CI	Code, collaboration on code	Creative work is a second-class attachment
Design tools	Collaborative canvases	Visual work, real-time design	Code is an export, not a resident
PM / trackers	Boards and issues	Coordinating work	Describe work without containing it
All-in-one docs	Workspace + databases	Docs, wikis, light DBs	Code & design files are guests
Portfolio tools	Showcase sites	Presenting finished work	No connection to the building of it

POSITIONING

The Everything Platform doesn't try to beat any one of these at its own game. It sits in the *seam between all of them* — the one place that treats the project, not the artifact type, as the unit.

06.2 Market sizing, bootstrapper's edition

A bootstrapped plan doesn't need a billion-dollar TAM; it needs enough reachable, willing payers to build a great living. The relevant math is small and concrete.

LAYER	DEFINITION	ROUGH SCALE	RELEVANCE
TAM	Everyone who codes and creates	Tens of millions	Inspiring, not actionable
SAM	Solo builders & micro-teams online	A few million	The people we could plausibly reach
SOM (yr 2)	Those we can reach build-in-public	~10–50k aware	Realistic top of funnel
Target (yr 2)	Weekly active builders	3–5k	The honest goal
Paying (yr 2)	Converted to paid	300–600	Where the living comes from

06.3 Competitive stance

The honest competitive truth: there is no single tool that does what this does, but there are ten tools that each do a slice of it well, plus the ultimate competitor — the status quo of stitching them together with tabs and willpower. The platform competes less against any product and more against inertia.

Why we won't be copied quickly

The incumbents are structurally committed to their worldview. A code platform can't make design a first-class citizen without rebuilding its core data model; a design tool can't make code resident without becoming a different company. Their strength is their cage.

Why we might be copied eventually

If the wedge proves real, a well-funded player could build toward it. The defense isn't secrecy — it's a head start, a loyal community, and the care that a solo studio can sustain and a growth-stage company cannot.

06.4 Moats a bootstrapper can actually build

- **Taste and care.** A product that obviously loves its users is hard to clone, because the cloners don't have the same relationship with them.
- **Community.** The people who grew up with the product in public become its defenders and its best feature requests.
- **Data gravity.** Once a builder's whole project history lives here, leaving means abandoning their memory. Switching cost is real and earned.
- **Speed of a single mind.** No meetings, no roadmap committees. The studio can turn a good idea into a shipped feature in days.

Compete in the seam, not the categories. Win the under-served solo builder the funded world ignores. Defend with care, community, and the data gravity of a project's full history. Treat inertia, not any product, as the real rival.

§07 / PRINCIPLES

Product principles

Seven rules the product obeys. When a decision is hard, these break the tie. They are written down so they can be enforced, and argued with.

07.1 The project is sacred

Everything attaches to a project. There is no feature that exists outside one, no orphan object floating in a global void. If a thing can't be explained as part of a project, it doesn't belong in the product.

07.2 History is never lost

Every meaningful change is recorded and recoverable. The timeline's value depends entirely on the user trusting it completely. We earn that trust by never, ever losing or silently rewriting history.

07.3 The free tier is the product

The free tier must be genuinely, lastingly useful — not a crippled trial. People should be able to do real work on it forever. The paid tier sells convenience and scale to people who already love the free one.

07.4 Less, but better

Every feature is a tax on the next person trying to understand the product. We add slowly, remove willingly, and treat a rejected feature as a small victory for clarity.

07.5 Speed is a feature

Fast software respects the user's time and a solo maker's attention. Performance is not an optimization phase; it's a constraint we design within from the first commit.

07.6 No dark patterns, ever

No manufactured urgency, no buried cancel buttons, no engagement bait. The user is the customer. A product for builders that disrespects builders deserves to fail.

07.7 Build in the open

Progress, decisions, and even mistakes are shared publicly. It's the cheapest marketing \$200 can buy, the strongest forcing function for quality, and the most honest way to earn a community.

THE TIE-BREAKER OF TIE-BREAKERS

When even the principles conflict, ask: "What would make Maya — the solo builder we're designing for — trust us more?" Trust compounds. Optimize for it.

§08 / ARCHITECTURE

Architecture

How the thing is actually built, chosen for one constraint above all others: a single maintainer running it for almost nothing until it has real usage.

08.1 Guiding constraints

- **Free until it works.** Every layer must have a free tier generous enough to run the product through its first thousands of users.
- **One person can hold it.** Boring, well-documented, popular technology beats clever, novel technology. The best architecture is the one the founder can debug at 2am alone.
- **Scale by usage, not by guess.** No infrastructure built for a load that doesn't exist yet. Add capacity when metrics, not anxiety, demand it.

08.2 The stack, by layer

LAYER	CHOICE (CATEGORY)	WHY	COST AT OUR SCALE
Frontend	Modern web framework	Loved, hireable, fast to build in	\$0 (open source)
Hosting	Edge / serverless platform	Generous free tier, zero ops	\$0 → low
Database	Managed Postgres (free tier)	Relational fits the data model; generous start	\$0 → low
Object storage	S3-compatible, pay-per-GB	Pennies per GB for files & assets	~\$1–\$5/mo later
Code/git	Git infra + lightweight server	Standard, portable, well-understood	\$0 (open source)
Search	Postgres full-text	Free, good enough until it	\$0

	first	isn't	
Auth	Open-source / free-tier auth	Don't roll your own; security matters	\$0 → low
Email	Transactional free tier	Low volume early; cheap to scale	\$0 → low
Analytics	Privacy-first, self-host option	Respect users; avoid heavy vendors	\$0

DELIBERATELY OMITTED

No Kubernetes, no microservices, no message queue, no separate data warehouse on day one. Each is a real tool for a real problem this product does not yet have. Adding them early would spend the scarcest resource — the founder's attention — on imaginary scale.

08.3 The core data model

Four objects do most of the work. Their relationships are deliberately simple, because the timeline's reliability depends on a model that's easy to reason about.

```
// the spine of the whole product
Project {
  id, name, visibility, owner_id, accent, created_at
}
Member {           // people on a project
  project_id, user_id, role: owner | editor | viewer
}
File {             // code, design, asset, doc – all files
  id, project_id, path, kind, current_version_id
}
Event {           // the timeline is just an ordered stream
of these
  id, project_id, actor_id, type, target_ref,
  decision_note?, group_id?, created_at
}
```

Everything else — tasks, comments, commits, assets — is either a kind of `File` or emits `Event`s into the timeline. The timeline is not a separate system; it is the ordered projection of every `Event` a project has ever produced. This is the single most important architectural decision in the product.

08.4 The timeline, technically

Every module that changes project state writes an append-only `Event`. The timeline view is a query over those events, filtered and grouped. Because events are append-only, history is structurally impossible to lose — satisfying principle 07.2. Grouping (a design change + its code change shown as one entry) is a `group_id` on related events, assigned when the relationship is known.

08.5 Security & privacy baseline

- Encryption in transit everywhere; encryption at rest via the managed providers.

- Least-privilege access; role checks enforced server-side, never trusted from the client.
- Private by default. Nothing is public unless the user explicitly makes it so.
- No selling data, no third-party ad trackers, privacy-first analytics only.
- A documented, practiced backup and restore of the database and object storage.

HONEST LIMIT

Enterprise-grade compliance (SOC 2, SSO, audit exports) is explicitly out of scope for years one and two. The plan serves people who don't require it yet, and revisits this only when a paying segment genuinely needs it.

08.6 Scaling plan, by user count

The point of this table is to *not* over-build. Each row is a trigger, not a schedule: do the work when the number is hit, not before.

USERS	WHAT BREAKS FIRST	THE FIX	EST. COST
0–500	Nothing	Ship on free tiers; ignore scale	\$0
500–2k	Storage creeps up	Lifecycle rules; cheap S3-compatible storage	~\$1–\$5/mo
2k–5k	DB free tier limits	Upgrade DB tier OR optimize queries	~\$10–\$25/mo
5k–15k	Search gets slow	Move from Postgres FTS to a dedicated index	~\$0–\$20/mo
15k+	Single-region latency	Add edge caching; consider read replicas	revenue-funded

THE COST PHILOSOPHY

Costs are designed to arrive *after* the revenue that pays for them. By the time the database needs upgrading, the paid tier should already cover it many times over. Infra that scales with usage is a feature, not a threat, when monetization scales alongside it.

08.7 Build vs. buy vs. borrow

CAPABILITY	DECISION	REASONING
Auth	Borrow (free-tier service)	Security-critical; never roll your own
Timeline / events	Build	This is the core IP; must own it

Git hosting	Borrow / open source	Standard, portable, solved
File storage	Buy (pennies)	Cheap, reliable, not worth building
Search	Build → buy later	Free Postgres FTS now; dedicated index at scale
Email	Buy (free tier)	Deliverability is hard; don't fight it
Payments	Buy (later)	Only when paid tier launches; standard processor

§09 / MODULES

Module catalog

The whole platform, specified part by part. Fourteen modules, each with purpose, features, user stories, acceptance criteria, and dependencies. The phase tag on each says when it ships in the 24-month plan.

09.0 Map of modules

#	MODULE	PHASE	PRIORITY	ONE-LINE
01	Projects	v1 · core	P0	The container for everything.
02	Unified files	v1 · core	P0	One place for every file a project contains, whether it's a code file, a design export, an image, or a document.
03	Timeline	v1 · wedge	P0	THE feature.
04	Code	v1 · core	P0	Version-controlled code that lives inside the project rather than in a separate silo.
05	Tasks	v1.1	P1	Lightweight work tracking that points at real work instead of floating above it.
06	Docs	v1.1	P1	Documents and notes that live in the project — briefs, specs, decision records, READMEs.
07	Comments & feedback	v1.1	P1	Feedback that lives on the thing it's about.

08	People & roles	v1.1	P1	A shared cast of people across every module, with roles simple enough that no one needs a permissions manual.
09	Assets & media	v1.2	P2	A proper home for the creative outputs — images, exports, brand assets, renders — with previews, organization, and history.
10	Publish & portfolio	v1.2	P2	Turn finished work into something shareable: a public project page, a simple portfolio, or a changelog the world can follow.
11	Search	v1.2	P2	Find anything in a project — a file, a decision, a comment, a commit message — from one box.
12	Integrations	v1.3	P2	Meet builders where they already are.
13	Time & invoicing	v1.3	P3	For the small studios (Priya) who bill clients: lightweight time tracking tied to projects and a simple invoice export.
14	Community & templates	ongoing	P2	A shared library of project templates, a public showcase of what people build, and the connective tissue of a community.

P0 = must exist for v1 P1 = early follow-on

P2/P3 = later / revenue

HOW TO READ A SPEC

Each module spec is a contract with the future builder (often: future-you). "Acceptance" lines are the bar for calling the module done — if they don't all pass, it isn't shipped.

MOD-01 • v1 • core

P0

Projects

PRI:P0

CORE

PHASE 1

The container for everything. A project holds code, files, docs, tasks, and people, and is the only top-level object a user creates. Getting this right makes every other module simpler; getting it wrong makes them all harder.

// FEATURES

- Create a project in under a minute, no required config
- Each project has a single shared timeline (see MOD-03)
- Project-level people & roles (owner, editor, viewer)
- Lightweight settings: name, description, visibility, accent color
- Archive / restore without destroying history
- Templates: start from blank, web app, brand kit, or portfolio

// USER STORIES

- As Maya I create a project and immediately start adding work, not filling forms
- As Priya I invite my collaborator and set them as editor in two clicks
- As Leo I start from a template so I'm not staring at a blank page

// ACCEPTANCE

- A new project is usable within 60 seconds of clicking 'New'
- Deleting is reversible for 30 days; nothing is truly gone by accident
- Visibility (private / link / public) is obvious and one toggle

Dependencies: None — this is the root object everything else attaches to.

MOD-02 • v1 • core

P0

Unified files

One place for every file a project contains, whether it's a code file, a design export, an image, or a document. Files are first-class objects with their own history, not attachments bolted onto something else.

// FEATURES

- Tree view that mixes code and creative files naturally
- Inline preview for images, common design exports, markdown, and code
- Drag-and-drop upload; paste-to-upload from clipboard
- Every file carries its own version history
- Tag files by kind (code / design / asset / doc) for filtering
- Object storage backing; pennies per gigabyte at the budget scale

// USER STORIES

- As Devin I see the mockup and the component file in the same tree
- As Renata I drop in a rendered frame and it previews instantly
- As Maya I find last week's logo version without hunting through chats

// ACCEPTANCE

- Common image and code files preview without download
- Upload of a 10MB file completes reliably on a normal connection
- Every file has a visible, restorable version history

Dependencies: MOD-01 (projects), object storage layer (see §08).

MOD-03 • v1 • wedge

P0

Timeline

PRI:P0

WEDGE

PHASE 1

THE feature. A single, unified history of every meaningful change in a project — a design edit, a code commit, a task closed, a decision noted — shown as one readable stream, attributed to people, grouped by intent. This is the reason to switch, and the spine the whole product hangs from.

// FEATURES

- Chronological stream of all change events across modules
- Filter by person, by module, by file, by date range
- 'Decisions' — pinned timeline notes that capture the why
- Group related changes (a design + its code) into one entry
- Git-log-style density for power users; readable summaries for everyone
- Permalink any moment to share 'here's where we were on Tuesday'

// USER STORIES

- As Devin I see the design change and the code that implemented it side by side
- As Maya I return after a week and the timeline tells me the whole story
- As Priya I send a client a link to the timeline and they get it instantly

// ACCEPTANCE

- Timeline renders 1,000 events without jank
- A design change and its linked code change can appear as one grouped entry
- Any timeline moment is shareable via stable permalink

Dependencies: MOD-01, MOD-02, and event hooks from every other module.

Code

PRI:P0

CORE

PHASE 1

Version-controlled code that lives inside the project rather than in a separate silo. Not a replacement for heavy git workflows on day one — a place for a solo builder's code to live with everything else and feed the timeline.

// FEATURES

- Git-backed repository per project
- Browse code with syntax highlighting in-app
- Commits flow into the unified timeline automatically
- Connect an existing external repo (mirror) as an alternative to native
- Simple in-browser edits for quick fixes
- File-level history shared with MOD-02

// USER STORIES

- As Maya my commits show up in the same timeline as my design changes
- As Devin I link a commit to the design decision that motivated it
- As Leo I see my code with nice highlighting without extra setup

// ACCEPTANCE

- A commit appears in the timeline within seconds, attributed correctly
- An external repo can be connected read-only without losing timeline events
- Syntax highlighting covers the top 15 languages out of the box

Dependencies: MOD-03 (timeline), git infrastructure (see §08).

MOD-05 · v1.1

P1

Tasks

PRI:P1

PHASE 2

Lightweight work tracking that points at real work instead of floating above it. A task can be tied to a file, a commit, or a decision, so closing it leaves a trace in the timeline rather than a dangling card.

// FEATURES

- Simple list and board views; no ceremony
- Tasks link to files, commits, or decisions
- Status changes emit timeline events
- Assignee, due date, and a single priority flag — nothing more
- Keyboard-first quick add
- Filter by assignee, status, linked module

// USER STORIES

- As Priya I track what's left on a client build without a heavyweight tool
- As Maya I close a task and it's recorded next to the commit that did it
- As Devin I open a task straight from the file it concerns

// ACCEPTANCE

- A task can be created with one keystroke and a title
- Closing a task is visible in the timeline with its linked work
- Board view handles 200 tasks without lag

Dependencies: MOD-01, MOD-03. Optional links to MOD-02, MOD-04.

MOD-06 · v1.1

P1

Docs

PRI:P1

PHASE 2

Documents and notes that live in the project — briefs, specs, decision records, READMEs. Markdown-first, simple, and woven into the timeline so a doc edit is part of the project's story.

// FEATURES

- Markdown editor with live preview
- Docs are files (MOD-02) with structured editing on top
- Link docs to tasks, files, and decisions
- Edits appear in the timeline
- Templates: brief, spec, decision record, changelog
- Public doc sharing via link

// USER STORIES

- As Maya I write the brief where the work will happen, not in a separate app
- As Devin I keep a decision record that links to the code it explains
- As Priya I share a clean spec doc with a client via link

// ACCEPTANCE

- Markdown renders accurately including code blocks and images
- A doc edit appears in the timeline as a single, sensible entry
- A public doc link works without an account

Dependencies: MOD-02, MOD-03.

MOD-07 · v1.1

P1

Comments & feedback

PRI:P1

PHASE 2

Feedback that lives on the thing it's about. A comment can be left on a file, a region of an image, a line of code, or a timeline entry — so feedback stops scattering across chat apps and starts accumulating in context.

// FEATURES

- Comment threads anchored to files, images, code lines, or events
- Region/pin comments on images and design exports
- Resolve / reopen, with timeline trace
- @mentions notify the right person
- Comment digests instead of constant pings
- Public comment mode for client review (optional)

// USER STORIES

- As Devin I pin feedback to the exact spot on the mockup
- As Priya my client leaves comments directly on the deliverable
- As Maya I review all open feedback in one place before shipping

// ACCEPTANCE

- A comment can be anchored to a precise image region
- Resolving a thread is reflected in the timeline
- Notifications are batched, never spammy by default

Dependencies: MOD-02, MOD-03, MOD-04.

MOD-08 · v1.1

P1

People & roles

PRI:P1

PHASE 2

A shared cast of people across every module, with roles simple

enough that no one needs a permissions manual. Attribution everywhere is what makes the timeline trustworthy.

// FEATURES

- Three roles: owner, editor, viewer
- Per-project membership; invite by link or email
- Consistent identity and avatar across all modules
- Activity attributed to people everywhere
- Guest access for client review (time-boxed)
- Leave / remove without breaking historical attribution

// USER STORIES

- As Priya I add a collaborator once and they have the right access everywhere
- As Maya I give a client a viewer link that expires
- As Devin I always know who changed what

// ACCEPTANCE

- Roles are understandable without documentation
- Removing a person preserves their historical attribution
- Guest links can be set to expire

Dependencies: MOD-01, auth layer (see §08).

MOD-09 • v1.2

P2

Assets & media

PRI:P2

PHASE 3

A proper home for the creative outputs — images, exports, brand assets, renders — with previews, organization, and history. The creative counterpart to the code module, treated with equal seriousness.

// FEATURES

- Gallery and grid views for visual assets
- Collections / boards within a project
- Versioned assets with visual diff (before/after)
- Color and basic metadata extraction
- Quick share of an asset or a collection
- Optimized delivery via storage/CDN

// USER STORIES

- As Renata I keep iterations of a piece with visual version history
- As Devin I grab the current logo export without asking anyone
- As Priya I share a brand collection with a client in one link

// ACCEPTANCE

- Visual diff shows previous vs current for an image asset
- A collection can be shared publicly via link
- Thumbnails generate reliably for common formats

Dependencies: MOD-02, MOD-03, storage/CDN.

MOD-10 • v1.2

P2

Publish & portfolio

PRI:P2

PHASE 3

Turn finished work into something shareable: a public project page, a simple portfolio, or a changelog the world can follow. Closing the loop from building in private to showing in public — which doubles as the product's growth engine.

// FEATURES

- Public project page generated from the project
- Opt-in 'building in public' changelog feed
- Simple portfolio that aggregates public projects
- Custom accent and cover; tasteful defaults
- 'Made with RMH' attribution (toggleable on paid)
- Shareable timeline embeds

// USER STORIES

- As Maya I flip a project public and have a clean page to share
- As Renata I show finished pieces without a separate portfolio tool
- As Leo I share my learning journey as a public changelog

// ACCEPTANCE

- A project can be made public with a presentable default page
- Public pages load fast and look intentional out of the box
- Attribution link drives measurable referral traffic

Dependencies: MOD-01, MOD-03, MOD-09.

MOD-11 · v1.2

P2

Search

PRI:P2

PHASE 3

Find anything in a project — a file, a decision, a comment, a commit message — from one box. As projects accumulate history, search is what keeps that history useful instead of buried.

// FEATURES

- Unified search across files, docs, tasks, comments, timeline
- Filter by module, person, date
- Keyboard shortcut to summon from anywhere
- Recent and pinned results
- Search within a single project or across all projects
- Fast, typo-tolerant matching

// USER STORIES

- As Maya I search 'auth decision' and find the doc, the commit, and the thread
- As Devin I jump to a file by name from anywhere with a shortcut
- As Priya I find everything about a client across projects

// ACCEPTANCE

- Search returns results in under 300ms at typical project size
- A keyboard shortcut opens search from any screen
- Results are grouped by module for scanability

Dependencies: All content modules; indexing layer (see §08).

MOD-12 · v1.3

P2

Integrations

PRI:P2

PHASE 4

Meet builders where they already are. A small, well-chosen set of

integrations — import an external repo, post to social on publish, receive webhooks — so the platform extends a builder's world instead of demanding they abandon it.

// FEATURES

- Connect/mirror an external code repository
- Outgoing webhooks on timeline events
- Share-to-social on publish
- Import files from common cloud storage
- Public read API for project data
- Embeddable timeline and asset widgets

// USER STORIES

- As Maya I keep my existing repo and still get unified history
- As Renata I auto-post when I publish a new piece
- As Devin I wire a webhook to my own tooling

// ACCEPTANCE

- An external repo can be mirrored without manual re-sync
- Webhooks fire reliably with documented payloads
- The read API is documented and rate-limited sensibly

Dependencies: MOD-04, MOD-03, MOD-10.

MOD-13 • v1.3

P3

Time & invoicing

PRI:P3

PHASE 4

REVENUE

For the small studios (Priya) who bill clients: lightweight time tracking tied to projects and a simple invoice export. A pragmatic revenue-relevant module that makes the paid tier worth more to the people most likely to pay.

// FEATURES

- Start/stop timers tied to a project or task
- Manual time entry and weekly summary
- Simple rate-based invoice generation (export)
- Per-client rollups across projects
- CSV export for accounting
- No payment processing — export only, to stay simple and cheap

// USER STORIES

- As Priya I track billable time where the work happens
- As Priya I export a clean invoice at month end
- As Maya I optionally track my own time to understand effort

// ACCEPTANCE

- A timer can be started from a task in one click
- An invoice export is clean enough to send a client as-is
- Time data exports to CSV without loss

Dependencies: MOD-01, MOD-05; paid-tier gating (see §15).

MOD-14 • ongoing

P2

Community & templates

PRI:P2

GROWTH

A shared library of project templates, a public showcase of what people build, and the connective tissue of a community. As much a growth engine as a feature: every shared template and

showcased project pulls new builders in.

// FEATURES

- Public template gallery; one-click 'use this template'
- Community showcase of public projects
- Featured builders and 'project of the week'
- Template submission and curation
- Lightweight following / favoriting
- Seasonal build challenges to drive activity

// USER STORIES

- As Leo I start from a community template and learn from it
- As Renata I get my piece featured and gain followers
- As Maya I publish a template and earn goodwill (and backlinks)

// ACCEPTANCE

- A template can be used to create a new project in one click
- The showcase updates without manual deploys
- Featuring a project measurably drives new signups

Dependencies: MOD-01, MOD-10. Curation is manual at first (by the founder).

§10 / DESIGN-SYSTEM

Design system

The visual and interaction language. A small, strict system is what lets one person ship a product that looks intentional instead of assembled.

10.1 Voice

Calm, precise, and a little warm. The product speaks like a thoughtful colleague, not a mascot. Labels say what happens ("Make public", not "Go live!"). Errors explain and offer a fix. Empty states invite action. Nothing shouts.

10.2 Color

TOKEN	ROLE	NOTE
Ink	Primary text & structure	Near-black, warm, never pure #000
Paper	Background	Warm off-white; easy on long sessions
Tang (accent)	The one bold color	Used sparingly: actions, the wedge, key marks
Teal	Secondary structure	Links, secondary emphasis, calm
Slate	Quiet text & meta	Captions, timestamps, the timeline's hum

10.3 Type

Three voices, mirroring the product itself: a grotesk display for headings (the studio's confident voice), a clean sans for body (clarity), and a monospace for data, code, and the timeline (the code voice). The pairing isn't decoration — it encodes the code-meets-craft thesis in the typography.

10.4 Layout & motion

- **Generous whitespace.** Density is earned only in the timeline and data views, where power users want it.
- **Hairline structure.** Thin rules and quiet dividers, not heavy boxes.
- **Restrained motion.** Motion clarifies state changes; it never performs. Reduced-motion preferences are always respected.
- **Keyboard-first.** Every common action has a shortcut; the product is fast for the hands that live on the keyboard.

10.5 Accessibility floor

Not a feature, a floor: sufficient contrast everywhere, visible keyboard focus, semantic structure, and full keyboard operability. A product for builders must be usable by all of them.

THE DESIGN NORTH STAR

When in doubt, remove something. The product should feel like a quiet, well-lit workshop — everything you need within reach, nothing on the bench you're not using.

§11 / BUDGET

The \$200 budget

Every dollar of the cash budget, named. The discipline of \$200 is the strategy, not a limitation to apologize for — it forces a product good enough to spread on its own.

11.1 The principle: cash vs. revenue-funded

The \$200 is the founder's **out-of-pocket cap before revenue**. It pays for the handful of things that genuinely cost money in the pre-revenue stretch. Everything that scales with usage — bigger database tiers, more storage, search infrastructure — is deliberately deferred until revenue covers it (see §08.6). The plan is built so the bills arrive *after* the income that pays them.

LINE ITEM	WHAT IT BUYS	TIMING	COST
Primary domain (.com, 2 yr)	The home address	M01	\$24.00
Secondary / brand domain (2 yr)	Brand protection + redirect	M06	\$20.00
Brand & logo	DIY mark + a few cheap assets	M01–M02	\$35.00
Font / icon licenses	Optional polish for the UI	M05	\$15.00
Object storage buffer	Pre-revenue file/asset overage	M02–M11	\$30.00
Transactional email	Low-volume sends before scale	M06–M11	\$12.00
Free-tier overflow buffer	A paid month if a free tier is outgrown early	M11	\$24.00
Launch & community	First build challenge, small	M16	\$20.00

	extras		
Contingency reserve	The thing you didn't plan for	—	\$20.00
Total cash budget		24 months	\$200.00

11.2 What this budget deliberately does NOT buy

No paid acquisition

Zero ad spend. Growth is earned through building in public, a genuinely useful free tier, and word of mouth. Every user in year one arrives for free.

No premature infrastructure

No upgraded database, no dedicated search, no CDN bill until usage — and the revenue that comes with it — demands them. See the scaling triggers in §08.6.

No tools you can do without

The toolchain is open source and free tiers. Paid productivity tools are a luxury this budget doesn't extend to until revenue makes them trivial.

No founder salary (yet)

The founder is paid in equity and runway, not cash, until month ~23. The plan's whole point is to reach the month where a real salary is covered by real revenue.

11.3 The real budget: time

Cash is the small constraint. The expensive resource is the founder's hours. The plan assumes a sustainable part-time pace — roughly 40–60 focused hours a month — not a heroic sprint that ends in burnout. Weekly shipping is the mechanism that keeps the time budget honest.

PHASE	MONTHS	EST. FOUNDER HOURS/MO	PACE
Foundation	M01–M06	50–60	Heaviest build; the core must exist
Habit & money	M07–M12	45–55	Build + first support load

Width	M13– M18	40–50	Build + community + support
Business	M19– M24	40–50	More support & ops, less net-new build

BUDGET SUMMARY

\$200 cash, spent on domains, a little storage, brand, and a couple of buffers. Everything that scales is funded by revenue, by design. The true investment is ~1,100 founder hours over two years — spent at a pace that can be sustained, not survived.

11.4 If revenue arrives (the happy reprojection)

Once the paid tier is live (M11), infrastructure costs become a small fraction of MRR. A product at 5,000 users with a few hundred payers spends perhaps \$30–\$80/month on infrastructure against many multiples of that in revenue. The \$200 was only ever needed to reach the month where the product pays for itself.

§12 / TIMELINE

24-month timeline

The operational heart of the plan. Twenty-four months, each with goals, a commit-style build log, what ships, the target metric, the cash spent, and the one risk to watch. Four phases of six months each.

12.0 The four phases at a glance

PHASE	MONTHS	THEME	EXIT CONDITION
P1 · Foundation	M01–M06	Narrow lovable core + v1 launch	Public v1, 50–100 weekly active
P2 · Habit & money	M07–M12	Complete the project + paid tier	First payers, free→paid loop works
P3 · Width	M13–M18	Assets, publish, search, community	Sticky, spreading, 150–300 paying
P4 · Business	M19–M24	Integrations, billing, scale, profit	Founder salary covered, \$0 raised

each dot = a build-log commit

cash spend shown per month; most months are \$0

HOW TO USE THIS

Treat each month as a sprint with one headline outcome (what "ships"). If a month slips, push the whole train back rather than skipping a deliverable — the order matters more than the dates.

§12.1 / PHASE 1

Phase 1 • Foundation

Build the narrow, lovable core: projects, unified files, the timeline wedge, and code. Ship a v1 that does one true thing better than any stitched-together stack. End the phase with real users, not a private demo.

SPAN	MONTHS	PHASE CASH	FOCUS
M01– M06	6	\$44	Build

M01 Foundations & the spine

FOUNDATION

Everything starts here. The temptation is to design the whole product; resist it. The only goal is a working spine — auth, the four-table model, and an empty timeline that proves the architecture holds. A public build-log post on day one starts the distribution flywheel before there's anything to show, which is exactly the right time to start.

GOALS

- Stand up the stack on free tiers (hosting, DB, auth, storage)
- Implement the core data model: Project, Member, File, Event
- First public build-log post: announce what's being built and why

BUILD LOG

- 
- 01a5207** init repo; deploy a 'hello' to the edge
 - 01a58f8** model Project + Member; auth wired
 - 01a5fe9** Event table append-only; first timeline render
 - 01a66da** write build-in-public post #1

DELIVERABLES

- init repo; deploy a 'hello' to the edge
- model Project + Member; auth wired
- Event table append-only; first timeline render
- write build-in-public post #1

DONE WHEN

A private deploy where you can create a project and see an empty timeline.

TARGET	CASH SPEND	PHASE	HOURS EST.
0 users · 1 build-log post · stack live	\$24 (primary domain,	P1 · build	50-60

2yr)

RISK TO WATCH • M01

Over-engineering the data model. Mitigation: ship the four-table model, resist more.

M02 Unified files

FOUNDATION

Files are where the code-meets-craft thesis first becomes visible. A code file and an image sitting in the same tree, both versioned, both feeding the timeline, is the first moment the product looks like nothing else. Keep preview support narrow; the rabbit hole of supporting every format will eat the month if allowed.

GOALS

- File upload, tree view, and inline preview (images, code, markdown)
- Per-file version history backed by object storage
- Files emit timeline events on add/change

BUILD LOG

- 
- 01a78d2** object storage wired; upload works
 - 01a7fc3** file tree mixes code + images
 - 01a86b4** inline preview for top formats
 - 01a8da5** file changes write Events

DELIVERABLES

- object storage wired; upload works
- file tree mixes code + images
- inline preview for top formats
- file changes write Events

DONE WHEN

You can drop any file into a project and see it previewed, versioned, and on the timeline.

TARGET

3-5 friendly
testers
invited

CASH SPEND

\$0

PHASE

P1 • build

HOURS EST.

50-60

RISK TO WATCH • M02

Preview rabbit hole. Mitigation: support common formats only; link-out for the rest.

M03 The timeline, for real

FOUNDATION

This is the most important month of the year. The timeline is the wedge, and the parts that make it special — grouping related changes, pinning decisions, sharing a permalink — are the parts no fragmented stack can replicate. If testers don't have an 'oh, that's the point' reaction this month, the thesis needs attention before anything else proceeds.

GOALS

- Readable, filterable timeline across files + system events
- Decisions: pinned timeline notes that capture the why
- Grouping: related changes can render as one entry

BUILD LOG

- 01a9f9d timeline filters by person/module/date
- 01aa68e decision notes pin to the stream
- 01aad7f group_id groups related events
- 01ab470 permalink any timeline moment

DELIVERABLES

- timeline filters by person/module/date
- decision notes pin to the stream
- group_id groups related events
- permalink any timeline moment

DONE WHEN

The wedge exists: one history of everything, shareable by link.

TARGET

10-15 testers
• first 'oh,
that's the
point'
reactions

CASH SPEND

\$0

PHASE

P1 • build

HOURS EST.

50-60

RISK TO WATCH · M03

Timeline feels generic. Mitigation: nail grouping + decisions, the parts nothing else has.

M04 Code module + closed beta

FOUNDATION

With the wedge in place, code joins the timeline and a small, hand-picked beta opens. Thirty users is plenty; the goal is depth of feedback, not breadth. Every beta user gets a real conversation. The external-repo mirror means people don't have to abandon their existing workflow to try the thing — lowering the cost of saying yes.

GOALS

- Git-backed code per project; commits flow into the timeline
- Connect an external repo as a read-only mirror
- Open a closed beta to ~30 hand-picked builders

BUILD LOG

- 
- 01ac668** native git repo per project
 - 01acd59** commits appear in timeline, attributed
 - 01ad44a** external repo mirror (read-only)
 - 01adb3b** closed beta invites go out

DELIVERABLES

- native git repo per project
- commits appear in timeline, attributed
- external repo mirror (read-only)
- closed beta invites go out

DONE WHEN

A solo builder's code and design changes share one timeline. Closed beta is live.

TARGET

~30 beta users
• first
weekly-active
cohort

CASH SPEND

\$0

PHASE

P1 • build

HOURS EST.

50-60

RISK TO WATCH • M04

Beta users churn silently. Mitigation: talk to every single one; weekly check-ins.

M05 Polish, onboarding, templates

FOUNDATION

Polish is where good products separate from demos, but polish is also infinite. Timebox it. The onboarding flow and starter templates are the highest-leverage work: they decide whether a stranger reaches the 'aha' before giving up. The performance pass protects the timeline from feeling sluggish as real projects accumulate events.

GOALS

- First-run experience: project created and useful in under a minute
- Three starter templates (blank, web app, brand kit)
- Performance pass: timeline handles 1,000 events smoothly

BUILD LOG

- 01aed33 60-second onboarding flow
- 01af424 starter templates ship
- 01afb15 perf pass on timeline render
- 01b0206 fix top 10 beta complaints

DELIVERABLES

- 60-second onboarding flow
- starter templates ship
- perf pass on timeline render
- fix top 10 beta complaints

DONE WHEN

A newcomer can go from signup to a real project fast, from a template.

TARGET

40-60 users •
activation
rate measured

CASH SPEND

\$0

PHASE

P1 • build

HOURS EST.

50-60

RISK TO WATCH • M05

Polish never ends. Mitigation: timebox to two weeks; ship, then move on.

M06 Public v1 launch

FOUNDATION

Launch month. Open signups, ship public project pages, post to the build-in-public channels that have been warming up for five months. Expect a spike and plan for the silence after it — the funnel instrumentation added here is what turns a vanity spike into a learnable cohort. The secondary domain locks down the brand.

GOALS

- Open public signups; launch on build-in-public channels
- Public project pages so people can share what they make
- Instrument the funnel: signup → first project → weekly active

BUILD LOG

- 01b13fe public signup opens
- 01b1aef public project page v1
- 01b21e0 launch posts + show-and-tell
- 01b28d1 funnel analytics live

DELIVERABLES

- public signup opens
- public project page v1
- launch posts + show-and-tell
- funnel analytics live

DONE WHEN

v1 is public. Anyone can build a project and share its timeline.

TARGET

150-300
signups · 50-
100 weekly
active

CASH SPEND

\$20
(secondary
domain,
2yr)

PHASE

P1 · build

HOURS EST.

50-60

RISK TO WATCH · M06

Launch spike then silence. Mitigation: have phase-2

features queued to retain.

§12.2 / PHASE 2

Phase 2 • Habit & money

Turn first users into a weekly habit, add the modules that make a project feel complete (tasks, docs, comments, people), and open a paid tier. End the phase with the first paying customers and a free→paid loop that works.

SPAN	MONTHS	PHASE CASH	FOCUS
M07– M12	6	\$24	Money

M07 Tasks + the paid tier groundwork

HABIT & MONEY

Phase two begins by making the project feel more complete and laying the rails for revenue. Tasks that link to real work (not floating cards) extend the timeline's reach. Billing is scaffolded but charges nobody yet; pricing is drafted and tested in conversations before a single price is published.

GOALS

- Lightweight tasks linked to files, commits, decisions
- Define free vs paid limits; build billing plumbing (no charge yet)
- Decide pricing (see §15) and soft-announce it

BUILD LOG

- 01b3ac9 tasks list + board ship
- 01b41ba tasks link to work; emit events
- 01b48ab billing integration scaffolded
- 01b4f9c pricing page drafted

DELIVERABLES

- tasks list + board ship
- tasks link to work; emit events
- billing integration scaffolded
- pricing page drafted

DONE WHEN

Tasks that point at real work, and the rails for charging money.

TARGET

300-500 users
• pricing
validated in
interviews

CASH SPEND

\$0

PHASE

P2 • money

HOURS EST.

45-55

RISK TO WATCH • M07

Pricing scares early fans. Mitigation: grandfather early users; keep free tier strong.

M08 Docs + decisions, deepened

HABIT & MONEY

Docs bring the brief, the spec, and the decision record into the project itself. Markdown only — the discipline that keeps this from becoming a doc tool in disguise. When a decision lives next to the code it explains and the design it shaped, the product's promise of unified context gets materially stronger.

GOALS

- Markdown docs as first-class project files
- Doc templates: brief, spec, decision record, changelog
- Public doc sharing via link

BUILD LOG

-  **01b6194** markdown doc editor ships
-  **01b6885** doc templates + linking
-  **01b6f76** public doc links
-  **01b7667** docs feed the timeline

DELIVERABLES

- markdown doc editor ships
- doc templates + linking
- public doc links
- docs feed the timeline

DONE WHEN

The brief, the spec, and the decisions live where the work is.

TARGET

500–750 users
• docs used in
30%+ of active
projects

CASH SPEND

\$0

PHASE

P2 • money

HOURS EST.

45–55

RISK TO WATCH • M08

Reinventing a doc tool. Mitigation: markdown only; lean on simplicity.

M09

Comments & feedback in context

HABIT & MONEY

Comments close the feedback loop: feedback anchored to the exact image region or code line, accumulating in context instead of scattering across chat. This month also holds the kill-criteria checkpoint (G2) — an honest look at whether the timeline is genuinely why people stay. A 'no' here is a signal to reshape, not to push harder.

- GOALS
- Threaded comments anchored to files, image regions, code lines, events
 - Resolve/reopen with timeline trace; @mentions; batched notifications
 - Mid-point thesis check: is the timeline the reason people stay?

- BUILD LOG
- 01b885f region comments on images
- 01b8f50 line comments on code
- 01b9641 resolve + mention + digests
- 01b9d32 run the M09 thesis review

- DELIVERABLES
- region comments on images
 - line comments on code
 - resolve + mention + digests
 - run the M09 thesis review

DONE WHEN

Feedback that accumulates on the work instead of scattering across chat.

TARGET	CASH SPEND	PHASE	HOURS EST.
700-1,000 users · kill-criteria checkpoint passed	\$0	P2 · money	45-55

RISK TO WATCH · M09

Notification spam. Mitigation: digests by default, never per-event pings.

M10 People, roles & guests

HABIT & MONEY

People and roles turn a solo tool into something small teams and client relationships can use. Three roles, no custom permissions, time-boxed guest access. The attribution audit ensures every action across every module is correctly tied to a person — the foundation of a timeline anyone can trust.

GOALS

- Clean three-role model (owner/editor/viewer) across all modules
- Invite by link or email; time-boxed guest access for clients
- Consistent identity & attribution everywhere

BUILD LOG

- 
- 01baf2a** roles enforced server-side
 - 01bb61b** invite links + email
 - 01bbd0c** expiring guest access
 - 01bc3fd** attribution audit across modules

DELIVERABLES

- roles enforced server-side
- invite links + email
- expiring guest access
- attribution audit across modules

DONE WHEN

Small teams and client review work without a permissions manual.

TARGET

1k-1.4k users
• first multi-person projects

CASH SPEND

\$0

PHASE

P2 • money

HOURS EST.

45-55

RISK TO WATCH • M10

Permissions complexity creep. Mitigation: exactly three roles, no custom perms.

M11 Open the paid tier

HABIT & MONEY

The paid tier opens. The first charge is a milestone worth celebrating publicly. Early supporters are grandfathered as thanks. The infra cost review confirms that margins are healthy at current scale before scaling further. Low conversion here is a value problem to investigate, not a price to cut in a panic.

GOALS

- Turn on billing; launch Pro tier (see §15)
- Cost review: confirm infra still under control at current scale
- First-dollar celebration, publicly

BUILD LOG

- 01bd5f5 paid tier live; first charge
- 01bdce6 grandfather early supporters
- 01be3d7 M11 infra cost review
- 01beac8 build-log: 'we have revenue'

DELIVERABLES

- paid tier live; first charge
- grandfather early supporters
- M11 infra cost review
- build-log: 'we have revenue'

DONE WHEN

The paid tier is open. The product makes money.

TARGET

First 10–30
paying
customers •
MRR > \$0

CASH SPEND

**\$24 (free-
tier
overflow
buffer)**

PHASE

P2 • money

HOURS EST.

45–55

RISK TO WATCH • M11

Conversion near zero. Mitigation: interview non-converters; adjust value, not price first.

M12 Year-one consolidation

HABIT & MONEY

A consolidation month closes year one. No new modules — just stability, debt paydown, documentation, and an honest public retro. The plan itself gets rewritten based on what twelve months actually taught. Skipping the retro to keep building is the temptation; the retro is a deliverable, not a luxury.

GOALS

- Stabilize, fix debt, document; no new modules this month
- Year-one retro published openly
- Re-plan phase 3 from what the year actually taught

BUILD LOG

- `01bfcc0` bug + debt burn-down
- `01c03b1` write the year-one retro
- `01c0aa2` update this very plan
- `01c1193` set phase-3 priorities from data

DELIVERABLES

- bug + debt burn-down
- write the year-one retro
- update this very plan
- set phase-3 priorities from data

DONE WHEN

A stable, profitable-trajectory product and an honest public retro.

TARGET

1.5k-2k users
• 30-60 paying
• MRR \$200-\$600

CASH SPEND

\$0

PHASE

P2 • money

HOURS EST.

45-55

RISK TO WATCH • M12

Skipping the retro to keep building. Mitigation: it's a deliverable, not optional.

§12.3 / PHASE 3

Phase 3 • Width & stickiness

Widen carefully: assets, publish/portfolio, search, and community. Each addition must deepen the habit, not dilute it. Turn the public side of the product into the growth engine. End the phase data-sticky and quietly spreading.

SPAN M13– M18	MONTHS 6	PHASE CASH \$20	FOCUS Width
---------------------	-------------	--------------------	----------------

M13 Assets & media as first-class

WIDTH & STICKINESS

Phase three widens the product, starting with assets as true first-class citizens — galleries, collections, visual diffs — the creative counterpart to the code module. Storage costs get their first real watching here; lifecycle rules and optimization keep the bill in line with the revenue now arriving.

GOALS

- Gallery/grid views, collections, visual version diff for images
- Optimized delivery via storage/CDN
- Creative outputs get the same seriousness as code

BUILD LOG

- 01c238b asset gallery + collections
- 01c2a7c visual before/after diff
- 01c316d CDN-backed delivery
- 01c385e thumbnails for common formats

DELIVERABLES

- asset gallery + collections
- visual before/after diff
- CDN-backed delivery
- thumbnails for common formats

DONE WHEN

The creative counterpart to the code module, fully realized.

TARGET

2k–2.6k users
• assets used
by creative
personas

CASH SPEND

\$0

PHASE

P3 • width

HOURS EST.

40–50

RISK TO WATCH • M13

Storage costs jump. Mitigation: lifecycle rules + optimization; watch the M13 metric.

M14 Publish & portfolio

WIDTH & STICKINESS

Publishing closes the loop from private building to public showing — and the showing is the growth engine. Public pages, opt-in changelogs, a simple portfolio, and tasteful attribution that brings new builders in. The danger is scope-creeping into a website builder; tasteful defaults and few knobs keep it focused.

GOALS

- Polished public project pages; opt-in build-in-public changelog feed
- Simple portfolio aggregating a builder's public projects
- 'Made with RMH' attribution as a growth loop

BUILD LOG

- 
- 01c4a56** public pages redesigned
 - 01c5147** changelog feed per project
 - 01c5838** portfolio surface
 - 01c5f29** attribution referral tracking

DELIVERABLES

- public pages redesigned
- changelog feed per project
- portfolio surface
- attribution referral tracking

DONE WHEN

Closing the loop: build in private, show in public — and the showing brings people.

TARGET

2.6k–3.3k
users • first
attribution-
driven signups

CASH SPEND

\$0

PHASE

P3 • width

HOURS EST.

40–50

RISK TO WATCH • M14

Portfolio scope creep into a site builder. Mitigation: tasteful defaults, few knobs.

M15 Search across everything

WIDTH & STICKINESS

As history accumulates, search is what keeps it useful instead of buried. Unified search across every module, summoned from anywhere by keyboard. Postgres full-text is free and good enough now; the plan to migrate to a dedicated index waits for the metric that actually demands it (§08.6), not for anxiety.

GOALS

- Unified search over files, docs, tasks, comments, timeline
- Keyboard summon from anywhere; grouped, typo-tolerant results
- Postgres full-text now; plan dedicated index for later

BUILD LOG

- 01c7121 unified search index
- 01c7812 keyboard launcher
- 01c7f03 grouped results by module
- 01c85f4 search within / across projects

DELIVERABLES

- unified search index
- keyboard launcher
- grouped results by module
- search within / across projects

DONE WHEN

History stays useful at scale: find anything in seconds.

TARGET

3.3k-4k users

CASH SPEND

\$0

PHASE

P3 • width

HOURS EST.

40-50

• search

adopted by

power users

RISK TO WATCH • M15

Search slow as data grows. Mitigation: measure; migrate index per §08.6 trigger.

M16 Community & templates v1

WIDTH & STICKINESS

Community becomes a deliberate growth engine: a template gallery that lowers the on-ramp, a showcase that celebrates what people build, and the first build challenge to spark activity. The empty-community problem is real — so the founder seeds the library and curates by hand until momentum carries it.

GOALS

- Public template gallery with one-click 'use this template'
- Community showcase of public projects; founder-curated 'project of the week'
- First seasonal build challenge

BUILD LOG

- 
- 01c97ec** template gallery + one-click use
 - 01c9edd** community showcase
 - 01ca5ce** project-of-the-week
 - 01cacbf** launch a build challenge

DELIVERABLES

- template gallery + one-click use
- community showcase
- project-of-the-week
- launch a build challenge

DONE WHEN

The community becomes a growth engine; templates lower the on-ramp.

TARGET

4k-4.8k

users ·

template-

driven

signups

measurable

CASH SPEND**\$20****(challenge/community)****PHASE**

P3 ·

width

HOURS

EST.

40-50

Empty-community problem. Mitigation: seed with founder templates; curate by hand.

M17 Stickiness & retention pass

WIDTH & STICKINESS

A retention month, not a feature month. A cohort deep-dive reveals who comes back and why, the top funnel leaks get sealed, and gentle re-engagement brings dormant projects back. Vanity metrics are explicitly set aside here in favor of weekly-active and retention — the numbers that tell the truth.

GOALS

- Retention deep-dive: who comes back weekly, and why
- Fix the top 3 drop-off points in the funnel
- Re-engagement: gentle digests of a project's recent timeline

BUILD LOG

- 01cbeb7 cohort retention analysis
- 01cc5a8 fix top funnel drop-offs
- 01ccc99 weekly project digest email
- 01cd38a win-back for dormant projects

DELIVERABLES

- cohort retention analysis
- fix top funnel drop-offs
- weekly project digest email
- win-back for dormant projects

DONE WHEN

A measurably stickier product, with the leaks in the funnel sealed.

TARGET

4.8k–5.5k
users • W4
retention
improving

CASH SPEND

\$0

PHASE

P3 • width

HOURS EST.

40–50

RISK TO WATCH • M17

Vanity metrics hide churn. Mitigation: track weekly-active and retention, not signups.

M18 Mid-plan consolidation

WIDTH & STICKINESS

The mid-plan consolidation mirrors month twelve: stabilize the wider surface, pay down the debt that rapid additions create, publish an eighteen-month retro, and revisit pricing with a year and a half of real data. Phase four's revenue priorities are set here, from evidence rather than hope.

GOALS

- Stabilize the wider surface; pay down debt from rapid additions
- 18-month public retro; revisit pricing with real data
- Decide phase-4 revenue priorities

BUILD LOG

- 
- 01ce582** stabilize + debt burn-down
 - 01cec73** write 18-month retro
 - 01cf364** pricing revisit with data
 - 01cfa55** set phase-4 plan

DELIVERABLES

- stabilize + debt burn-down
- write 18-month retro
- pricing revisit with data
- set phase-4 plan

DONE WHEN

A broad, stable platform and a clear-eyed plan for turning it into a living.

TARGET

5k-6k users •
150-300 paying
• MRR \$1.2k-
\$2.5k

CASH SPEND

\$0

PHASE

P3 • width

HOURS EST.

40-50

RISK TO WATCH • M18

Breadth outpacing quality. Mitigation: a consolidation month is mandatory, not optional.

§12.4 / PHASE 4

Phase 4 • Business

Make it a real, self-sustaining business: integrations, time & invoicing for paying studios, scale where metrics demand it, and revenue that pays the founder a living wage. End month 24 able to choose the next move from strength.

SPAN	MONTHS	PHASE CASH	FOCUS
M19– M24	6	\$0	Profit

M19 Integrations

BUSINESS

Phase four turns the product into a business. Integrations let the platform extend a builder's existing world — webhooks, share-to-social, a documented read API — rather than demanding they abandon their other tools. The surface is kept small, versioned, and rate-limited so one person can maintain it.

GOALS

- External repo mirror hardened; outgoing webhooks on timeline events
- Share-to-social on publish; import from common cloud storage
- Documented public read API + embeddable widgets

BUILD LOG

- 
- 01d0c4d** webhooks on events
 - 01d133e** share-to-social on publish
 - 01d1a2f** cloud storage import
 - 01d2120** public read API + embeds

DELIVERABLES

- webhooks on events
- share-to-social on publish
- cloud storage import
- public read API + embeds

DONE WHEN

The platform extends a builder's existing world instead of demanding they leave it.

TARGET

6k-7k users ·
integrations
used by power
segment

CASH SPEND

\$0

PHASE

P4 · profit

HOURS EST.

40-50

RISK TO WATCH · M19

API maintenance burden. Mitigation: small, versioned, rate-limited surface only.

M20 Time & invoicing for studios

BUSINESS

Time tracking and invoice export give the studios most likely to pay a concrete reason to upgrade. Deliberately export-only: no payment processing, to avoid becoming an accounting product. For a team billing clients, this single module can justify the Studio tier on its own.

GOALS

- Timers tied to projects/tasks; manual entry; weekly summaries
- Rate-based invoice export; per-client rollups; CSV out
- Position as the reason small studios upgrade

BUILD LOG

- 01d3318 project/task timers
- 01d3a09 invoice export (no payments)
- 01d40fa per-client rollups
- 01d47eb studio upgrade prompts

DELIVERABLES

- project/task timers
- invoice export (no payments)
- per-client rollups
- studio upgrade prompts

DONE WHEN

Billing studios get tangible paid-tier value; the segment most likely to pay, served.

TARGET

7k-8k users •
studio
conversion
lifts

CASH SPEND

\$0

PHASE

P4 • profit

HOURS EST.

40-50

RISK TO WATCH • M20

Becoming an accounting tool. Mitigation: export only; never process payments.

M21 Scale where the metrics say

BUSINESS

Now, and only now, infrastructure scales — driven by the \$08.6 triggers actually being hit, funded entirely by revenue rather than the \$200. A practiced restore, a reliability dashboard, and edge caching where latency demands it. Premature scaling was avoided for twenty months precisely so this month could be cheap and confident.

GOALS

- Act on \$08.6 triggers actually hit (DB tier, search index, caching)
- Reliability pass: backups practiced, error budgets watched
- All scale spend is now revenue-funded, not from the \$200

BUILD LOG

- 01d59e3 upgrade what metrics demand
- 01d60d4 practice a real restore
- 01d67c5 edge caching where needed
- 01d6eb6 reliability dashboard

DELIVERABLES

- upgrade what metrics demand
- practice a real restore
- edge caching where needed
- reliability dashboard

DONE WHEN

Infrastructure that matches real usage — paid for by revenue, not anxiety.

TARGET

8k-9k users ·
p95 latency
healthy

CASH SPEND

**\$0 (infra
now
revenue-
funded)**

PHASE

P4 · profit

HOURS EST.

40-50

Premature scaling. Mitigation: only act on triggers from §08.6, never on fear.

M22 Monetization deepening

BUSINESS

With a year of behavioral data, monetization gets refined: an annual plan for cash-flow smoothing, an optional team plan, and dunning to recover involuntary churn. Price changes remain slow and rare — value comes first, always. The goal is a model tuned by evidence, not a clever pricing trick.

GOALS

- Refine tiers with a year of data; add an annual plan
- Optional studio/team plan for multi-seat
- Reduce involuntary churn (dunning, reminders)

BUILD LOG

- 01d80ae annual plan + discount
- 01d879f team/studio plan
- 01d8e90 dunning + payment recovery
- 01d9581 pricing experiment readout

DELIVERABLES

- annual plan + discount
- team/studio plan
- dunning + payment recovery
- pricing experiment readout

DONE WHEN

A monetization model tuned by real behavior, not guesses.

TARGET

9k-10k users ·
MRR \$3k-\$5k

CASH SPEND

\$0

PHASE

P4 · profit

HOURS EST.

40-50

RISK TO WATCH · M22

Over-optimizing price vs product. Mitigation: value first; price changes are slow & rare.

M23 Sustainability & the team question

BUSINESS

The sustainability check: does revenue cover costs many times over and a real founder salary? With that answered, the big question — stay solo, hire the first person, or raise — gets decided from strength. Runbooks are written so the studio survives the founder taking a week off, addressing the single-point-of-failure risk head-on.

GOALS

- Confirm revenue covers costs many times over and a founder salary
- Decide: stay solo, hire #1, or raise — from strength
- Document everything so the studio isn't a single point of failure

BUILD LOG

- 01da779 financial review: salary covered?
- 01dae6a write the 'what next' memo
- 01db55b ops runbooks
- 01dbc4c first contractor scoped (if warranted)

DELIVERABLES

- financial review: salary covered?
- write the 'what next' memo
- ops runbooks
- first contractor scoped (if warranted)

DONE WHEN

A business that pays its founder and could survive them taking a week off.

TARGET

10k-12k users

• 400-600

paying • MRR

\$4k-\$7k

CASH SPEND**\$0****PHASE**

P4 • profit

HOURS EST.

40-50

RISK TO WATCH · M23

Founder burnout. Mitigation: this is why the plan ships weekly, not heroically.

M24 Year-two close & the open choice

BUSINESS

Year two closes with a public retro that's honest about what didn't work, a year-three thesis, and a reinvestment plan made from a position of choice rather than necessity. The destination was never a valuation — it was a self-sustaining product, a living wage, and the freedom to decide what comes next. That's the win.

GOALS

- 24-month public retro; celebrate the milestone openly
- Set the year-three thesis (stay small / grow / raise)
- Reinvest from a position of choice, not necessity

BUILD LOG

- 01dce44 write the 24-month retro
- 01dd535 set year-three thesis
- 01ddc26 reinvestment plan
- 01de317 thank the community publicly

DELIVERABLES

- write the 24-month retro
- set year-three thesis
- reinvestment plan
- thank the community publicly

DONE WHEN

A self-sustaining product, a living wage, and a free choice about what comes next.

TARGET

Goal: self-sustaining • founder salary covered • \$0 outside capital

CASH SPEND

\$0

PHASE

P4 • profit

HOURS EST.

40-50

RISK TO WATCH • M24

Declaring victory too early. Mitigation: the retro is honest

about what didn't work too.

§13 / GATES

Milestones & gates

Phases don't advance on the calendar; they advance on evidence. Each gate is a question that must be answered "yes" before spending the next six months.

13.1 The gates

GATE	AT	THE QUESTION	IF 'NO'...
G1 · Wedge real?	end M06	Do users cite the timeline as why they'd switch?	Re-shape the wedge before widening
G2 · Habit formed?	end M09	Are 300+ people weekly-active? (kill-criteria)	Pivot the core, don't add features
G3 · People pay?	end M12	Is free→paid > 2% and MRR climbing?	Fix value before adding modules
G4 · Sticky & spreading?	end M18	Is retention up and growth partly organic?	Deepen, don't broaden, in P4
G5 · A real business?	end M24	Does revenue cover costs and a salary?	Reassess: smaller, slower, or stop

THE DISCIPLINE

A gate you always pass isn't a gate. Each is written so the honest answer could be "no" — and a "no" is a signal to change course cheaply, not to push harder into a wall.

13.2 Quarterly rituals

- **Public retro every quarter.** What shipped, what didn't, what the numbers say. Builds trust and forces honesty.
- **Re-read this plan every quarter.** Rewrite the parts the last three

months proved wrong. The plan is a hypothesis, not scripture.

- **Founder check-in every quarter.** Is the pace sustainable? Burnout is the single largest risk to a solo plan; it's tracked like any other metric.

13.3 Definition of done for v1

So "v1" can't quietly expand: v1 is done when a new user can sign up, create a project, add code and design files, see them in one unified timeline, capture a decision, and share a public link — all within their first session, on the free tier, with no confusion. Anything beyond that is v1.1 or later.

Go-to-market

With \$0 for ads, distribution has to be earned. The strategy is a single coherent motion: build in public, give generously, let the product show itself.

14.1 The motion

Every part of the go-to-market plan is something a solo founder can do for free, and most of it is a byproduct of building the product well. The flywheel: build in public → attract curious builders → they make things on the platform → they share those public projects → those projects pull in more builders.

CHANNEL	WHAT IT IS	COST	WHY IT FITS A \$200 PLAN
Build in public	Share progress, decisions, numbers	\$0	The product's story is the marketing
Public projects	Users' own work, shared	\$0	Every shared project is an ad with attribution
Templates & showcase	Community library + features	\$0	Lowers on-ramp; creates reciprocity
Founder writing	Essays on the code/craft seam	\$0	Earns the right audience, slowly
Community presence	Be genuinely useful where builders gather	\$0	Trust compounds; never spam
Word of mouth	A product worth recommending	\$0	The only channel that scales for free

14.2 The first 100 users (by hand)

The first hundred users are recruited personally, one conversation at a

time, from the build-in-public audience and communities of builders. Each is interviewed. This doesn't scale — which is the point. The first hundred teach the product what it needs to be before any attempt at scale would just amplify the wrong thing.

14.3 From 100 to 1,000

Once the product earns genuine "I love this" reactions, the public-projects loop and templates begin to compound. A v1 launch moment (M06) on build-in-public channels provides a spike; the retention work in P2 keeps the people the spike delivers.

14.4 Positioning & message

THE PITCH, ONE LINE

"One home for your code and your craft — with a single history of how it all came together."

The message leads with the pain everyone recognizes (the tax of stitching tools together) and the relief of the wedge (one timeline for everything). It never leads with a feature list. People don't switch tools for features; they switch to stop hurting.

14.5 Launch sequencing

MOMENT	WHEN	GOAL
Soft beta	M04	30 hand-picked builders; learn
Open beta	M05	Activation tuning; templates
Public v1	M06	First real spike; instrument funnel
Paid launch	M11	First revenue; grandfather fans
Community launch	M16	Templates + showcase as engine
Year-two milestone	M24	Public retro; set what's next

14.6 What we won't do for growth

- No buying email lists, no cold spam, no growth hacks that erode trust.
- No fake urgency or manufactured scarcity. Ever (principle 07.6).
- No chasing a viral moment at the expense of the people already here.

§15 / PRICING

Pricing & monetization

Simple, fair, and built on a free tier that's genuinely useful forever. The paid tier sells scale and convenience to people who already love the product.

15.1 The tiers

TIER	PRICE	FOR	KEY LIMITS / VALUE
Free	\$0	Maya, Leo, anyone	Generous projects & storage; full timeline; public pages
Pro	~\$8/mo	Solo builders who've outgrown free	Unlimited projects, more storage, priority features
Studio	~\$18/mo	Priya & small teams	Multi-seat, guest review, time & invoicing, client pages

PRICING PHILOSOPHY

Prices are indicative and validated with users before launch (M07 interviews, M11 launch). The free tier is never crippled to force upgrades; it's the front door and the best marketing. Early supporters are grandfathered as a thank-you.

15.2 Why these numbers

- **~\$8 Pro** sits below the threshold where a solo builder agonizes over it, and roughly replaces one of the tools they'll cancel.
- **~\$18 Studio** is trivially justified for a team billing clients — the time & invoicing module alone pays for it.
- **Annual plans** (added M22) offer a modest discount and smooth cash flow.

15.3 What is never paywalled

The wedge. The unified timeline, decisions, and core history are free forever. Charging for the thing that makes the product trustworthy would poison the well. Money comes from scale (more projects, more storage, more seats) and from the studio-specific tools, never from the core promise.

15.4 Unit economics, roughly

METRIC	ESTIMATE	NOTE
Infra cost / active user	pennies/mo	Free tiers + cheap storage
Gross margin	very high	Software with near-zero marginal cost
Support cost	founder's time	The real cost; managed by good docs
CAC	~\$0	Earned distribution, no ad spend
Payback	immediate	No acquisition cost to recover

Growth model

Three loops that compound. None of them costs money; all of them are byproducts of building the product well.

16.1 Loop 1 — Build in public

The founder shares progress; curious builders follow; some try the product; their feedback makes it better; better progress is more worth sharing. The loop's flywheel is honesty — sharing the failures as well as the wins is what earns the trust that converts.

16.2 Loop 2 — Public projects

Users make things; they share the public project page or timeline; viewers see a tasteful "Made with RMH" attribution; some become users; they make and share things. This is the strongest loop because it scales with usage and requires nothing from the founder once built (MOD-10, MOD-14).

16.3 Loop 3 — Templates & community

A user publishes a template; newcomers start from it; some of them publish their own; the library deepens; a deeper library attracts more newcomers. Curation (founder-led at first) keeps quality high enough that the loop doesn't degrade.

THE COMPOUNDING BET

None of these loops is explosive. All of them compound. A bootstrapped product doesn't need a viral spike; it needs loops that turn each cohort of happy users into the next cohort, quietly, for years.

16.4 The funnel

STAGE	DEFINITION	LEVER
Aware	Saw a post, project, or template	Build in public; public projects
Signed up	Created an account	Clear value prop; low friction
Activated	Created a real project, saw the timeline	60-second onboarding; templates
Weekly active	Came back and built more	The product being genuinely useful
Paid	Upgraded to Pro/Studio	Outgrew free; needed scale or studio tools
Advocate	Shared, referred, templated	A product worth recommending

§17 / METRICS

Metrics & KPIs

What gets measured, what gets ignored, and the one number that matters most. Vanity metrics are explicitly named so they can be resisted.

17.1 The north star

NORTH STAR METRIC

Weekly Active Builders (WAB) — the number of people who do real work in a project in a given week. Not signups, not page views. The number that says people actually use the thing.

17.2 The metric tree

TIER	METRIC	WHY IT MATTERS
North star	Weekly Active Builders	Real, recurring usage
Acquisition	Signups, signup→activation %	Top of funnel & first-run quality
Activation	% creating a real project wk 1	Did they reach the 'aha'?
Retention	W1 / W4 / W12 cohort retention	The truth about product-market fit
Revenue	MRR, free→paid %, churn	Is it a business?
Health	Infra cost/user, p95 latency	Is it sustainable & fast?
Founder	Hours/week, burnout check	Is the maker okay? (it's a real KPI)

17.3 Targets by checkpoint

CHECKPOINT	WAB	PAYING
M06 · v1 launch	50–100	0
M12 · year one	300–500	30–60
M18 · mid plan	800–1,200	150–300
M24 · year two	1,500–2,500	400–600

17.4 Vanity metrics to ignore

Total signups (without activation), social followers (without conversion), feature count, and any number that goes up while WAB and retention go down. These flatter the ego and lie about the business.

§18 / FINANCIALS

Financial projections

The bootstrap math. Conservative, base, and optimistic paths — all of which lead to the same destination: a product that pays for itself.

18.1 Costs are tiny by design

Because the stack runs on free tiers until usage forces upgrades, total cost stays near zero for a long time. The \$200 cash budget covers the pre-revenue period; after that, infrastructure is a small, revenue-funded line.

PERIOD	USERS	MONTHLY INFRA COST	FUNDED BY
M01–M06	0–300	~\$0	The \$200 budget
M07–M12	300–2k	~\$5–\$25	Mostly the \$200; early revenue
M13–M18	2k–6k	~\$25–\$60	Revenue
M19–M24	6k–12k	~\$60–\$150	Revenue (a fraction of MRR)

18.2 Three revenue paths (M24)

SCENARIO	PAYING USERS	AVG REVENUE/USER	APPROX MRR
Conservative	~250	~\$9	~\$2,250
Base	~450	~\$11	~\$4,950
Optimistic	~650	~\$12	~\$7,800

Even the conservative path reaches self-sustaining. The optimistic path is a real, modest livelihood — not a unicorn, by design. The plan's success criterion was never a valuation; it was independence.

18.3 The break-even logic

Break-even (covering infra + tools) arrives early — likely within a few months of the paid launch, because costs are so low. The meaningful threshold is later: the month revenue covers a founder salary, projected around M22–M24 in the base case. Everything between break-even and salary-covered is reinvested into the product and a small buffer.

18.4 Sensitivity — what moves the outcome

LEVER	IMPACT	CONTROLLABLE?
Free→paid conversion %	Largest single driver of MRR	Partly — via value, not pressure
Churn	Quietly erodes everything	Yes — via retention work (P3)
WAB growth	Sets the ceiling on payers	Partly — via the growth loops
Infra cost discipline	Protects margin	Yes — via §08.6 triggers
Founder pace	Determines if the plan survives	Yes — the most important lever

18.5 The financial philosophy

IN ONE LINE

Spend almost nothing, give away the core, charge fairly for scale, let costs arrive after the revenue that pays them, and define success as independence rather than size.

This is a plan to build a small, durable, self-owned business — the kind that a funded competitor can't easily replicate, because they're playing a different game with different numbers. The \$200 isn't a handicap. It's the whole strategy.

§19 / RISK

Risk register

What could go wrong, how likely, how bad, and the specific plan for each. A risk you've named and planned for is half-handled.

#	RISK	LIKELY	IMPACT	MITIGATION
R1	Wedge isn't compelling enough to switch	Med	High	Validate by M06; reshape, don't widen (G1)
R2	Founder burnout	Med	Critical	Sustainable pace; weekly ships; quarterly check-in
R3	Free→paid conversion too low	Med	High	Interview non-converters; improve value first
R4	A funded competitor copies the wedge	Low	Med	Head start, community, care, data gravity
R5	Infra costs outrun revenue	Low	Med	\$08.6 triggers; costs deferred behind revenue
R6	Launch spike then silence	High	Med	Retention features queued before launch
R7	Scope creep dilutes the product	High	Med	Principles 07.4; mandatory consolidation months
R8	Solo = single point of failure	Med	High	Document everything; runbooks by M23
R9	Data loss erodes core trust	Low	Critical	Append-only events; practiced backups

				(M21)
R10	Market is too niche to sustain	Low	High	SOM math (\$06.2); even conservative path works

THE META-RISK

The largest risk to any solo, bootstrapped plan isn't competition or technology — it's the founder running out of energy or honesty. Everything in this plan (weekly shipping, public retros, gates that can say "no") is designed to protect against exactly that.

19.1 Risk response playbook

For the three risks most likely to actually bite, a pre-decided response so panic doesn't make the decision in the moment.

R1 • wedge

If the timeline doesn't land

Don't add modules to compensate. Interview ten weekly users, find the real "stay" reason, and rebuild the wedge around it. A wrong wedge fixed fast beats a wide product no one needs.

R2 • burnout

If the pace breaks the founder

Cut scope, not corners. Drop a month's net-new build, ship only fixes, and rest. The plan explicitly permits a "quiet month." A living plan beats a dead founder.

R6 • silence

If launch fizzles

Expected, not feared. Return to recruiting users by hand, ship the queued retention features, and treat the spike's leftovers as the real cohort to learn from.

19.2 What would make us stop

An honest plan names its own off-ramp. If, by month 12, WAB is below ~150, conversion is near zero, and user interviews reveal no compelling reason to stay — the kindest move is to wind down gracefully, open-source what's useful, thank the community, and apply the lessons to the next thing. Stopping well is a skill, not a failure.

BUT THE BASE CASE

None of this is the expectation. The expectation is a slow, compounding climb to a small, durable business. The risk register exists so that if reality disagrees, the response is already written — calm, pre-decided, and honest.

Appendices

The reference material you'll actually open: checklists, a glossary, and the working notes that keep the plan executable.

30.1 Launch checklist (v1)

- ❑ Signup → first project flow works in under 60 seconds
- ❑ Three starter templates ship and are tested
- ❑ Timeline renders 1,000 events without jank
- ❑ Public project pages look intentional by default
- ❑ Funnel analytics instrumented end to end
- ❑ Backups verified with a practiced restore
- ❑ Build-in-public launch posts drafted and scheduled
- ❑ First 30 beta users personally thanked and interviewed

30.2 Paid-launch checklist

- ❑ Billing integration tested end to end (and the cancel flow especially)
- ❑ Free vs Pro vs Studio limits clearly communicated
- ❑ Early supporters grandfathered
- ❑ Dunning / failed-payment handling in place
- ❑ Pricing page honest, no dark patterns
- ❑ Infra cost review confirms healthy margin

30.3 Weekly operating rhythm

DAY	FOCUS
Mon	Plan the week's one headline outcome
Tue–Thu	Build the headline feature

Fri	Ship + write the build-in-public update
Sat	Talk to users; triage feedback
Sun	Rest. (Non-negotiable — see R2.)

30.4 Glossary

TERM	MEANING
Builder	Someone who both codes and creates; the core user
The wedge	The unified timeline — the one feature worth switching for
Project	The container for all of a piece of work; the core object
Timeline	The ordered stream of every change event in a project
Decision	A pinned timeline note capturing the why behind a change
WAB	Weekly Active Builders — the north star metric
Build in public	Sharing progress, decisions, and numbers openly
Gate	An evidence-based checkpoint between phases (G1–G5)
MRR	Monthly Recurring Revenue
Border tax	The hidden cost of moving work across fragmented tools

30.5 Module timeline index

MODULE	SHIPS IN	PHASE
Projects, Files, Timeline, Code	M01–M06	P1
Tasks, Docs, Comments, People	M07–M10	P2
Assets, Publish/Portfolio, Search, Community	M13–M16	P3
Integrations, Time & Invoicing	M19–M20	P4

30.6 Open questions (to revisit quarterly)

- Is "The Everything Platform" the final name, or a working title to replace before v1?

- At what user count does support time threaten the build pace — and what's the plan?
- Does a mobile companion ever earn its place, or stay web-first indefinitely?
- When (if ever) does taking on a collaborator beat staying solo?

CLOSING NOTE

This document is a hypothesis dressed as a plan. Its job is done not when it's admired but when it's marked up, argued with, and rewritten by a founder who learned something the plan got wrong. Build the narrow thing, ship it weekly in public, give the core away, charge fairly for scale, and let two years of compounding turn \$200 and a point of view into something that pays for itself. Then choose what's next — from strength.

— END OF DOCUMENT —

RMH STUDIOS • THE EVERYTHING PLATFORM • MASTER PLAN v0.9

RMH Studios Confidential draft • v0.9 pp. 71

